

A Voice RAG System, and Everything It Got Wrong First

Building a Hindi speech-to-answer retrieval pipeline that holds a sub-200 ms retrieval budget over 103,068 chunks — and a record of how measurement overturned nearly every default along the way.

Corpus 103,068 chunks **Retrieval P50** 11.54 ms **MRR@10** 0.551

Status deployed, HF ZeroGPU

CONTENTS

00	Abstract	07	Results
01	Motivation	08	What measurement overturned
02	Background	09	Failure analysis
03	Dataset	10	Deployment
04	Architecture	11	Limitations
05	Algorithms	12	Conclusion
06	Methodology	13	Résumé and interview

00 Abstract

What was built, and what the measurements showed.

This project implements an end-to-end voice question-answering system over a multilingual passage corpus: a user speaks a question in Hindi, the audio is transcribed, relevant passages are retrieved from a 103,068-chunk index, and an answer is returned that is grounded in what was retrieved — or explicitly refused when nothing supports one.

The engineering target was a query-time retrieval budget under 200 ms. The final system achieves **11.54 ms at P50 and 53.57 ms at P100** on an RTX 4060, measured over 200 queries after warm-up. Retrieval quality on 1,219 gold-labelled queries reaches **MRR@10 0.551** and **Recall@5 0.744** in its strongest configuration. A cross-encoder abstention gate reduces ungrounded answers on genuinely unanswerable questions from **100% to 14.7%**.

The more transferable result is methodological. Every significant default in the system was chosen by ablation rather than convention, and most of those ablations contradicted the conventional choice: semantic chunking lost to plain sentence splitting, hybrid dense+sparse retrieval lost to dense-only, and an abstention threshold that scored 0.93 balanced accuracy against easy negatives turned out to block only 26% of realistic ones. Three separate tuned values reversed when the corpus grew from 5,000 to 100,000 chunks, establishing that in this system *every hyperparameter is corpus-size dependent*.

01 Motivation

The problem, and why the obvious framing of it is wrong.

Retrieval-Augmented Generation exists to solve a specific failure of language models: they answer confidently from parametric memory, which is frozen at training time and cannot be audited. RAG addresses this by retrieving source passages at query time and constraining the answer to them, which makes answers both updatable and traceable.

Adding *voice* input raises the stakes on latency, because speech is a conversational medium and people tolerate far less delay in conversation than in typing. Adding a *low-resource language* — Hindi, in Devanagari script — raises the stakes on retrieval quality, because most retrieval components are tuned and benchmarked on English.

The 200 ms requirement, and why it cannot mean what it appears to

The original specification called for the full pipeline to complete in under 200 ms. Measurement shows this is not achievable by any system built on a hosted speech-to-text API, and it is worth being precise about why rather than quietly redefining the target.

MEASURED END-TO-END VOICE PATH, WARM PROCESS		
STAGE	MEASURED	NATURE
Speech-to-text (Sarvam)	1,139–2,566 ms	network round trip
Retrieval (embed + search)	13.6 ms	local compute
Generation (extractive)	32 ms	local compute
Total	1,185–2,612 ms	—

MEASURED FINDING

The speech-to-text network hop alone is roughly **100× the entire local retrieval segment**. No pipeline that calls a hosted STT service reaches 200 ms end-to-end; the round trip alone exceeds it fivefold.

The honest resolution is to scope the budget to what it can meaningfully describe — the **query-time retrieval segment**: embedding the query, searching the index, and optional reranking. That segment is local, is genuinely fast, and is the part an engineer actually controls. Both numbers are reported together throughout, and the harness times every stage separately so the split is always visible rather than hidden inside an average.

This framing decision matters beyond bookkeeping. A system that reports only the flattering number is not measuring; it is marketing. The instrumentation exists specifically so the unflattering number cannot be avoided.

02 Background

The concepts this system is built from, defined plainly.

This section defines the technical vocabulary used throughout. If you already know what a bi-encoder is, skip to §3.

Embeddings and dense retrieval

An **embedding** is a fixed-length vector of numbers that represents a piece of text's meaning. A model is trained so that texts with similar meaning land close together in that vector space. "What is a corporation?" and "A corporation is a legal entity..." end up near each other even though they share few words.

Dense retrieval exploits this: embed every passage once at index time, embed the query at search time, and return the passages whose vectors are closest. Closeness is measured by **cosine similarity** — the cosine of the angle between two vectors, ranging from -1 to 1 . Because the vectors here are normalised to unit length, cosine similarity reduces to a plain dot product, which is why it is fast.

This is called a **bi-encoder** arrangement: query and passage are encoded *independently*, never seeing each other. That independence is what makes it scalable — passage vectors are computed once, offline — and it is

also its main weakness, which §5 returns to.

Sparse retrieval and BM25

Sparse retrieval matches on literal words rather than meaning. **BM25** is the standard algorithm: it scores a document higher when it contains the query's rare words, more times, in a shorter document. "Sparse" refers to the representation — a vector with one dimension per vocabulary word, almost all zeros.

Dense and sparse fail differently, which is why combining them is conventional. Dense handles paraphrase and synonymy; sparse handles exact strings like names, codes and numbers that an embedding may blur. §7 shows that on this corpus the combination did not, in fact, help.

Chunking

Passages must be split into retrievable units. The unit matters: too large and the embedding averages several topics into a vague vector; too small and it loses the context needed to answer. Four strategies were implemented and compared — fixed-size windows, sentence packing, recursive separator splitting, and semantic (embedding-based) boundary detection. §7 reports which won.

Approximate nearest neighbour search and HNSW

Comparing a query against all 103,068 vectors exhaustively is *exact* but linear in corpus size. **Approximate nearest neighbour (ANN)** search trades a small amount of recall for a large speed gain. **HNSW** — Hierarchical Navigable Small World — is the ANN structure used here, described in §5.

Cross-encoder reranking

A **cross-encoder** takes the query and one passage *together* and outputs a relevance score. Because it can attend across both texts, it is markedly more accurate than a bi-encoder — and far too slow to run over a whole corpus. The standard pattern is a two-stage cascade: retrieve a cheap candidate set with the bi-encoder, then rerank those candidates with the cross-encoder.

Guardrails, grounding and abstention

An answer is **grounded** when it is supported by the retrieved context. The characteristic RAG failure is a fluent answer to a question the corpus cannot support — the retriever returns its best match regardless of whether that match is any good, and generation proceeds on it. **Abstention** is the guardrail: detect that nothing retrieved is relevant enough and decline. §7 measures what this buys, and §8 records how badly it was calibrated the first time.

03 Dataset

MS MARCO-XI, and the label that makes the whole evaluation possible.

The corpus is `ai4bharat/MSMARCO-XI` — a multilingual translation of MS MARCO, a large passage-ranking benchmark derived from real Bing queries. The full dataset is roughly 11.5M rows and 55 GB, so it is streamed and capped rather than downloaded whole.

Each row contains a query, a gold answer, and a `passages` field holding parallel lists: the English passages, their translations, and an `is_selected` flag.

THE CRITICAL AFFORDANCE

`is_selected == 1` marks the passage a human judged to actually answer the query. This label is what converts "the retrieval seems good" into a measurable claim — it makes Recall@k, MRR and nDCG computable rather than guessed. Every quantitative result in this report rests on it.

Two indexes, deliberately separate

The system builds two distinct indexes, and conflating them would invalidate the evaluation:

- The **serving index** (`ingest.py` → `data/index/`) is the corpus the live demo answers from — 10,000 rows, 103,068 chunks.
- The **evaluation index** (`eval.py`, in memory) is rebuilt per run and tracks which chunk ids came from gold passages, so metrics can be computed. Reported results use 2,000 rows → 21,657 chunks and 1,219 gold-labelled queries.

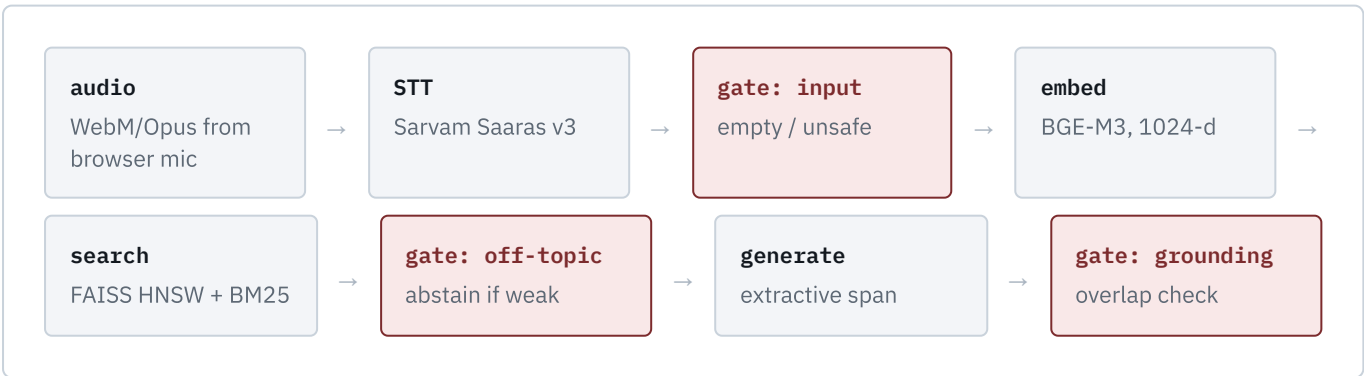
Dataset traps encountered

Two cost real time and are worth recording, because both fail quietly rather than loudly:

- **The documented per-language configs do not exist.** `load_dataset(DATASET_ID, "hi")` fails — the only available config is `default`. The per-language data lives in separate parquet files, loaded directly by URL.
- **There is no `en` language code.** English is not a separate file; it lives *inside* each language's parquet as `English_passages` / `Eng_Query` / `Eng_Answer`. Switching only the passage field leaves Hindi queries pointed at English passages — which does not raise an error, it silently returns poor retrieval. The `USE_ENGLISH` flag moves all three fields together and the config refuses to start on a half-applied switch.

04 Architecture

Stage order, and what each stage is responsible for.



Component choices

STACK, AND THE REASON FOR EACH SELECTION		
COMPONENT	CHOICE	REASON
Speech-to-text	Sarvam Saaras v3	Indic-specialised; strong Devanagari accuracy
Embedder	BAAI/bge-m3	Multilingual incl. Indic; 1024-d; fits 8 GB VRAM
Vector index	FAISS HNSW	0.57 ms over 103k vectors; scales to millions
Lexical index	bm25s	Sparse-matrix BM25; 652× faster than rank_bm25 (§8)
Reranker	bge-reranker-v2-m3	Pairs with BGE-M3; off by default on latency grounds
Generation	Extractive	~30 ms and grounded by construction (§7)
Serving	FastAPI + Gradio	Gradio is the only SDK HF ZeroGPU supports

The harness

The pipeline is not a script of sequential calls; it is a staged orchestrator with a defined contract. Each stage is timed independently into a `StageTiming` record, network stages carry retries with exponential backoff and an enforced timeout, inputs and outputs are Pydantic models rather than loose dicts, and failures degrade rather than propagate — if LLM generation fails, extractive generation answers instead.

The per-stage timing is not incidental. It is what made the BM25 regression in §8 diagnosable in minutes: the numbers pointed directly at the offending component instead of requiring a bisect.

DESIGN NOTE

Graceful degradation has a cost that §9 documents: a fallback that silently catches every exception will hide a component that is *completely broken*. The degradation is correct; the missing piece was testing the primary path directly rather than through its own fallback.

05 Algorithms

The mathematics actually running, and what each formula is doing.

BM25 — Best Matching 25

BM25 scores how well a document matches a query on literal word overlap. For query Q and document D :

$$\text{score}(D, Q) = \sum_{q_i} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D|/\text{avgdl})}$$

Three ideas are encoded here, and the third caused a real bug in this project:

- **IDF (inverse document frequency)** — rare words carry more signal than common ones. Matching "corporation" says more than matching "the".
- **Term-frequency saturation** — the k_1 term makes repeated occurrences yield diminishing returns. Ten mentions of a word are not ten times more relevant than one.
- **Length normalisation** — the $b \cdot |D| / \text{avgdl}$ term penalises long documents, on the reasoning that a match in a short document is more meaningful.

WHERE THIS BIT

Length normalisation *rewards* very short documents. Degenerate two-word chunks such as "क्या है?" ("what is?") scored **BM25 14.5** and outranked genuine passages — becoming the top-1 result the answer was generated from. The fix was not to BM25 but to chunking: merge fragments below 80 characters. Dense nDCG@10 rose **26%** (0.416 → 0.525).

Cosine similarity

Dense relevance is the cosine of the angle between query and passage vectors:

$$\cos(A, B) = (A \cdot B) / (\|A\| \cdot \|B\|)$$

Vectors are L2-normalised at index time, so $\|A\| = \|B\| = 1$ and this collapses to a dot product — one fused multiply-add per dimension, which is why FAISS can search 103,068 vectors in 0.57 ms.

HNSW — Hierarchical Navigable Small World

HNSW is a multi-layer proximity graph. Each vector is a node connected to its near neighbours; higher layers are sparse and act as express lanes, lower layers dense and local.

A search enters at the top layer, greedily walks to the closest node it can find, then drops a layer and repeats — coarse to fine, like a skip list in metric space. Complexity is roughly $O(\log N)$ rather than the $O(N)$ of exhaustive search.

Two parameters trade recall against cost: M (32 here) is the neighbour count per node, and $efSearch$ (128) is the candidate-list breadth during search. Higher values find more true neighbours and cost more time and memory. The "approximate" is real — HNSW can miss a true nearest neighbour — but at 0.57 ms per search over 103k vectors the trade is overwhelmingly favourable.

Score fusion: min-max versus RRF

Combining dense and sparse scores requires making them comparable, since cosine lives in $[0,1]$ and BM25 is unbounded. Two approaches were implemented and compared.

Min-max normalisation rescales each score list into $[0,1]$ over the candidate set, then blends by weight α :

```
norm(s) = (s - min) / (max - min)
fused   = α · norm(dense) + (1 - α) · norm(bm25)
```

STRUCTURAL FLAW

Min-max normalisation pins the best candidate at exactly **1.0 regardless of how weak it actually is**, because the maximum always maps to 1. The scale is relative to whatever candidates happened to surface, not absolute. This same flaw independently broke the abstention guardrail (§8) — a threshold on a fused score could never fire, because gibberish also scored 1.0000.

Reciprocal Rank Fusion (RRF) avoids this by combining *ranks* rather than scores:

$$\text{RRF}(d) = \sum \frac{1}{(k + \text{rank}_i(d))} \quad k = 60$$

Because rank is ordinal, RRF is immune to one signal's score distribution dominating. It is the standard remedy, and §7 reports what happened when it was tested.

Evaluation metrics

METRICS COMPUTED, AND WHAT EACH IS SENSITIVE TO		
METRIC	DEFINITION	SENSITIVE TO
Recall@k	$ \text{gold} \cap \text{top-}k / \text{gold} $	gold <i>count</i> — see caveat
Hit@k	1 if any gold in top-k, else 0	presence only
MRR@10	1 / rank of first gold	position of first hit
nDCG@10	DCG / ideal DCG, log-discounted	full ranking quality
Token F1	harmonic mean of token precision/recall	answer wording overlap

nDCG deserves unpacking since it is the least intuitive. Discounted Cumulative Gain sums the relevance of results with a logarithmic positional discount — a hit at rank 1 contributes $1/\log_2(2) = 1.0$, at rank 4 only $1/\log_2(5) \approx 0.43$. Dividing by the ideal ordering's DCG normalises to [0,1], making scores comparable across queries with differing numbers of gold passages.

METRIC CAVEAT THAT SHAPED EVERY COMPARISON

Recall@k divides by the number of gold *chunks*, so it is not comparable across chunking strategies that split the same passage differently. Semantic chunking emits ~40% more chunks, inflating its denominator. **Hit@k and MRR@10 are denominator-free** and are the fair cross-configuration comparisons. Conclusions in §7 are stated on those, which makes the reported margins smaller — and correct.

Extractive generation

The default generator runs no language model. It embeds the query and each sentence of the top passage, then returns the two highest-cosine sentences in their original order. This is ~30 ms, and grounded by construction — the output is verbatim source text, so it cannot hallucinate. It cannot synthesise or paraphrase either, which §7 quantifies against the LLM alternative.

06 Methodology

How the system was actually built, in the order it happened.

The working rule throughout: **every quality claim must be backed by a number from the evaluation harness, and when a component is added, its evaluation is added with it.** That rule is what produced the negative results in §7 — they were not sought, they emerged because the alternatives were measured rather than assumed.

PHASE 1 — BASELINE

Scaffold the pipeline, resolve the dataset loading traps (§3), build a 500-row smoke index, and establish the first latency and quality numbers. Deliberately small: the point was a working end-to-end path, not good results.

PHASE 2 — ABLATIONS

Sweep the choices that were guesses. Four chunking strategies; dense, sparse and hybrid retrieval; the fusion weight α across a grid; reranking on and off. Each measured on the same gold-labelled query set so results are comparable.

PHASE 3 — GUARDRAILS AND FAITHFULNESS

Measure what abstention buys by running the identical query set with guardrails enabled and disabled. This required a deliberate design decision: the harness gained a `guardrails_enabled` flag purely so the unguarded baseline could be measured. A claim about reducing hallucination is meaningless without it.

PHASE 4 — SCALE

Grow the corpus 19× to 103,068 chunks and re-run everything. This phase produced the most valuable findings in the project, because it revealed that conclusions from Phase 2 did not survive it (§8).

PHASE 5 — DEPLOYMENT AND PACKAGING

Ship a live demo, then the packaging a portfolio project needs: license, CI, unit tests, documentation.

Experimental discipline

Three practices did the real work:

- **Warm up before timing.** The first query pays model load and CUDA initialisation — roughly 10 s. Left in the timed loop it lands in P100 and reports a cold start as tail latency. Uncorrected, the headline was **9,845 ms**; corrected, **141 ms** on the same build.

- **Use denominator-free metrics for cross-configuration claims**, for the reason in §5.
 - **Test the alternative explanation before accepting a negative result.** Applied in §7's hybrid-retrieval finding, and it is the difference between a conclusion and an assumption.
-

07 Results

Every number here is reproducible from the repository.

RETRIEVAL P50 11.54 ms · 103k chunks	RETRIEVAL P100 53.57 ms · target 200	MRR@10 0.551 1,219 queries	RECALL@5 0.744 hybrid + rerank
UNGROUNDED 14.7% from 100%			

7.1 Latency

BENCHMARK.PY --N 200 · RTX 4060 · WARMED UP			
STAGE	P50	P70	P100
embed	9.49	9.73	40.23
retrieve	2.06	2.33	14.68
retrieval_total	11.54	11.86	53.57

P100 of 53.57 ms against a 200 ms target — and notably *faster* than the 19× smaller corpus was before the BM25 fix (P50 14.00 → 11.54 ms). Enabling the cross-encoder adds ~332 ms at P50, which breaks the budget outright; that is why reranking is off by default and is presented as an accuracy option rather than the standard path.

7.2 Retrieval quality

EVAL.PY --ROWS 2000 · 1,219 GOLD-LABELLED QUERIES · 21,657-CHUNK INDEX						
CONFIGURATION	R@1	R@5	HIT@5	MRR@10	NDCG@10	ANS F1
BM25 only	0.136	0.402	0.424	0.259	0.311	0.196
Dense only	0.312	0.701	0.727	0.494	0.562	0.281
Hybrid $\alpha=0.6$	0.212	0.612	0.641	0.400	0.478	0.240
Hybrid + rerank	0.364	0.744	0.773	0.551	0.610	0.293

WHY THESE NUMBERS ARE LOWER THAN AN EARLIER RUN

An earlier 500-row evaluation (5,592 chunks, 248 queries) scored MRR@10 **0.606** and Hit@5 **0.871**. Quadrupling the haystack drops those to 0.551 and 0.773. Nothing regressed — the earlier task was simply easier. Retrieval metrics are a property of *corpus size as much as retriever quality*, so an MRR quoted without its corpus size conveys very little. The larger-corpus numbers are the credible ones and are what this report reports.

7.3 Chunking strategy

ALL FOUR STRATEGIES · SAME 248 GOLD QUERIES · HYBRID + RERANK					
STRATEGY	CHUNKS	R@5	HIT@5	MRR@10	NDCG@10
sentence	5,592	0.843	0.871	0.606	0.673
fixed	5,602	0.832	0.875	0.612	0.669
recursive	5,958	0.827	0.879	0.604	0.663
semantic	7,975	0.676	0.839	0.578	0.574

OVERTURNED ASSUMPTION

The sophisticated option lost. Semantic chunking — embed every sentence, split at topic boundaries — came *last* on every metric while costing roughly **10× more to build**, because it runs the embedder over each sentence before indexing. Plain sentence packing won.

Reading this honestly: the R@5 gap (0.843 vs 0.676) overstates the case, since semantic emits ~40% more chunks and inflates its own denominator. On the denominator-free metrics semantic still loses (Hit@5 0.839 vs 0.871; MRR@10 0.578 vs 0.606) — so the conclusion holds, on the smaller margin.

The top three sit within noise of each other. `sentence` is the default because it wins nDCG@10, produces the smallest index, and never cuts mid-sentence — which matters specifically because the extractive generator returns whole sentences.

7.4 Does hybrid retrieval help?

FUSION-WEIGHT SWEEP · A=1.0 IS PURE DENSE · 1,219 QUERIES					
α	0.0	0.6	0.8	0.9	1.0
MRR@10 (min-max)	0.259	0.401	0.458	0.485	0.496
MRR@10 (RRF)	0.259	0.425	0.464	0.483	0.495

OVERTURNED ASSUMPTION

Adding BM25 at any weight made retrieval worse. Hybrid dense+sparse retrieval is close to received wisdom in RAG; on this corpus at this scale it lost to dense-only.

Crucially, the likelier alternative explanation was tested first. The suspicion was that *min-max fusion* was at fault rather than BM25 — it has the structural flaw described in §5. RRF, the standard remedy, was implemented and swept. It *did* help mid-range ($\alpha=0.6$: 0.425 vs 0.401) but still peaked at pure dense. So both are true: the fusion was mildly suboptimal *and* BM25 genuinely does not help here.

A negative result that survives its own strongest counter-argument is worth considerably more than one that was never challenged. BM25 remains wired in and one config change away, because the answer is corpus-dependent rather than permanent — it *did* help at 5,442 chunks.

7.5 What the guardrails buy

FAITHFULNESS.PY --N 150 · SAME QUERIES, GUARDRAILS ON VS OFF · 103K CHUNKS				
GATE	ANSWERS ANSWERABLE	ANSWERS UNANSWERABLE	ADDED LATENCY	
Guardrails OFF	100.0%	100.0%	—	
Cosine (fast path)	88.7%	52.7%	0 ms	
Cross-encoder	76.7%	14.7%	~332 ms	

Ungrounded answers on genuinely unanswerable questions fall from **100% to 14.7%** — an 85% reduction. Both columns are reported deliberately: a guardrail that abstained on everything would post a perfect 0% hallucination rate while being useless. The cost is real — 23 points of coverage on answerable questions.

The negative set is what makes this measurement meaningful. It is *not* hand-written nonsense; it is real MS MARCO queries drawn from rows *after* the indexed slice — natural, on-domain questions whose passages simply were never indexed. §8 records what happened when easier negatives were used instead.

7.6 Why a cross-encoder

DISCRIMINATIVE POWER ON ANSWERABLE VS UNANSWERABLE QUERIES				
DISCRIMINATOR	ANSWERABLE	UNANSWERABLE	GAP	BEST BAL. ACC
Bi-encoder cosine	0.669	0.590	0.079	0.710
Cross-encoder	0.860	0.350	0.510	0.813

The cross-encoder separates the classes **6.5× more widely**, because it performs full query–document attention instead of comparing independently-encoded vectors. When reranking is already enabled its score is free — the computation has happened.

A cascade was built to get that quality cheaply — cosine first, cross-encoder only for ambiguous cases — and **rejected on measurement**: the distributions overlap so heavily that 71% of queries land in the ambiguous

band, costing ~236 ms to buy *less* accuracy (0.793) than simply always reranking (0.813). There is no cheap version of this.

7.7 Generation mode

EXTRACTIVE VERSUS HOSTED LLM GENERATION		
MODE	LATENCY	CHARACTER
extractive (default)	~30 ms	verbatim source sentences; grounded by construction
llm (sarvam-105b)	21 / 40 / 79 s	synthesises a definition; better prose

The LLM answers are genuinely better — it defines a concept rather than quoting a sentence. At a 40-second median it is unusable interactively, so extractive remains the default. The decision is recorded with its numbers rather than as a preference.

08 What measurement overturned

The central finding: in this system, no tuned value survived a change of scale.

Three separate defaults reversed when the corpus grew from ~5,000 to ~100,000 chunks. Not one was a bug; each was a correct conclusion drawn at a scale that no longer applied.

TUNED VALUES BEFORE AND AFTER THE 19× SCALE-UP			
PARAMETER	SMALL CORPUS	AT SCALE	WHAT CHANGED
CHUNK_STRATEGY	semantic (assumed)	sentence	semantic worst on every metric, 10× build cost
MIN_DENSE_SCORE	0.50	0.58 + CE gate	block rate fell 80% → 52.7%
HYBRID_ALPHA	0.9	1.0	BM25 helped, then stopped helping

8.1 The guardrail that could not fire

The off-topic gate originally thresholded the *fused* retrieval score. Because fusion applies min-max normalisation over the candidate set (§5), the best candidate is pinned near 1.0 for *every* query. Measured directly:

FUSED SCORE FOR THE TOP RESULT, BY QUERY TYPE		
QUERY	FUSED SCORE	DENSE COSINE
Real Hindi question	0.9726	0.5689
<code>asdf qwerty zxcv</code>	1.0000	0.4818

OVERTURNED ASSUMPTION

Pure gibberish scored a **perfect 1.0000** — higher than a genuine question. The guardrail was **mathematically incapable of firing**. It had been in the codebase, documented as a safety feature, doing nothing. The fix was to threshold the dense cosine, which is absolute and comparable across queries rather than relative to whatever candidates surfaced.

8.2 The threshold validated on easy cases

The rebuilt gate was calibrated against gibberish: "asdf qwerty", "what is the capital of Mars". It scored **0.93 balanced accuracy** and looked finished.

Measured against realistic negatives — real MS MARCO queries whose passages were never indexed — that same threshold blocked only **26%** of them.

TOP DENSE COSINE BY NEGATIVE TYPE		
NEGATIVE SET	MAX COSINE	BLOCKS REALISTIC NEGATIVES
Gibberish (n=10)	0.434	26%
Real out-of-index queries (n=150)	0.719	80%

Gibberish sits so far from everything in embedding space that it validates almost any threshold. The 0.93 was measuring the *test set*, not the guardrail. Recalibrating on hard negatives moved the threshold 0.50 → 0.58 and the block rate from 26% to 80%.

TRANSFERABLE LESSON

Calibrating a safety mechanism on easy cases gives you a number, not a guarantee. The calibration script now draws negatives from validation rows past the indexed slice, and prints the trivial-gibberish score purely as a reminder of how easy the old test was.

8.3 The threshold that did not survive scaling

At 5,442 chunks the recalibrated cosine gate blocked 80% of unanswerable queries. At 103,068 chunks the same threshold blocks **52.7%**.

Nothing regressed. A larger corpus means any query — answerable or not — is likelier to find a plausible near-match, so scores rise for both classes. Gibberish that scored 0.427 at 5.4k scores 0.499 at 103k.

TRANSFERABLE LESSON

An abstention threshold is only valid for the corpus size it was tuned on. This is invisible until you scale, which is the argument for scaling *before* believing a safety number. Re-running the faithfulness measurement after any corpus change is now standard practice in this repository, documented at the point of use.

09 Failure analysis

Four failures that reported success. These are the expensive kind.

9.1 A 3.7× wrong extrapolation

Scaling to 103k chunks broke the latency budget: retrieval P50 91 ms, P100 525 ms. Profiling apportioned the blame immediately, thanks to per-stage timing:

RETRIEVAL COST DECOMPOSITION AT 103K CHUNKS		
COMPONENT	PER QUERY	SHARE
FAISS HNSW search	0.57 ms	<1%
rank_bm25 get_scores	63.75 ms	99%

Before the swap, a linear extrapolation from the 5.4k corpus predicted **17 ms**. The real cost was 63.75 ms — **3.7× worse**, because `rank_bm25` is pure Python and allocates a full-corpus array per query term, degrading superlinearly.

Replacing it with `bm25s`, which is sparse-matrix backed, took the same query from 63.75 ms to **0.06 ms — a 652× speedup** — at 0.9983 rank correlation with the previous implementation. The 19×-larger corpus ended up *faster* than the small one had been.

9.2 A dead feature hidden by graceful degradation

LLM generation mode had never actually run. It was broken: the configured model `sarvam-m` had been deprecated and the API returned HTTP 400. Nobody noticed, because `generate()` catches every exception and falls back to extractive.

TRANSFERABLE LESSON

Graceful degradation hides breakage. The feature was dead and the system looked healthy. Fallback paths must be tested *directly*, bypassing the try/except — otherwise you are testing the fallback, not the feature.

9.3 A timeout that was decorative

Fixing 9.2 exposed the next layer. `STAGE_TIMEOUT_S = 15` was declared in config and **wired to nothing** — which is how a 79-second API call ran to completion. A timeout that is advertised but never enforced is worse than none, because it buys false confidence.

Enforcing it exposed a third layer: tenacity was retrying *timeouts*, so a slow model cost $3 \times 15 \text{ s} = 46 \text{ s}$ before degrading — indistinguishable from a hang. Timeouts now fail fast into degradation; connection resets and 5xx still retry.

9.4 Five deploy failures, three of which exited zero

Deploying to Hugging Face ZeroGPU took five attempts. The failures are worth recording because most reported success:

1. **402 on repo creation.** Hardware must be specified *at creation*; a Gradio Space cannot be created on free CPU and then switched, because creation itself is rejected. There is no first step to take.
2. **Frontmatter rejected after staging.** A 61-character description against a 60-character limit, validated server-side *after* the Space was created and 555 MB staged.
3. **The 430 MB index silently dropped.** Output read `Found 31 files ... Committing 30/30`, exit code 0. Cause: the repository's own `.gitignore`, which `huggingface_hub` feeds to the preupload API — and which carries `*.faiss` to keep build artifacts out of git. Correct for the repo, catastrophic for the Space, where the index *is* the payload.
4. **Removing it from the upload was insufficient.** The resolution order ends with "the `.gitignore` already hosted on the Hub", and an earlier run had published one. It had to be deleted from the Space.
5. **The demo exhausted its daily GPU quota in ~8 queries.** `@spaces.GPU` wrapped the entire pipeline, so the Space held an accelerator throughout the 1–2.5 s *network* call to STT. Restructured so only retrieval holds the GPU.

TRANSFERABLE LESSON

Three of these five **exited 0 while failing**. The deploy script now verifies every staged file actually arrived and fails loudly naming what is missing. On this platform a green exit code is not evidence — and a deploy that succeeds while dropping your index is the worst available failure mode, because everything looks fine until the runtime logs say otherwise.

10 Deployment

Getting it hosted, and why the obvious route was closed.

The system is live on Hugging Face Spaces running ZeroGPU, deployed by a single command. Getting there required working around a platform change that most documentation has not caught up with.

The free tier is not what it was

As of August 2026, Hugging Face gates compute Spaces behind a paid plan: *"Static Spaces are free for everyone. Gradio and Docker Spaces run on compute and require a paid plan to create."* Creating a Docker Space on a free account returns HTTP 402.

A DISTINCTION THAT SAVES A WASTED DAY

The gate is the **Space type**, not resource usage. Shrinking or quantising the model does *not* unlock a free Docker Space — an intuitive theory that is entirely wrong. The free path that does exist is **Gradio + ZeroGPU**: two Spaces free for accounts in good standing, and it grants a *real* GPU, so the measured latency figures roughly hold rather than needing a CPU caveat.

Architecture of the deployment

- **Gradio front end** (`app_gradio.py`) — ZeroGPU supports only the Gradio SDK. Its `@spaces.GPU` decorator is a no-op off-Spaces, so the identical file runs locally.
- **GPU scope minimised** — only retrieval and generation hold the accelerator. STT, a network call, runs outside it. This is the difference between ~8 demo queries per day and a usable allowance.
- **Prebuilt index shipped** — 555 MB via LFS. Building at startup would exceed the timeout.
- **Pinned torch** — ZeroGPU supports a specific version range; leaving it unpinned lets pip install one outside that window and silently break GPU allocation.
- **Secrets at creation** — the API key is set via `space_secrets`, never baked into an image.

Operational reality

The free ZeroGPU tier allows **5 minutes of GPU per day, shared across all visitors**. A demo link that returns a quota error reads as broken to someone who does not know that, so a recorded GIF sits in the README as a hedge and both surfaces state the limit explicitly.

Engineering support

The repository carries the packaging a portfolio project needs: MIT licence, GitHub Actions CI, and 25 unit tests. The metrics were deliberately extracted into a stdlib-only module (`src/metrics.py`) so CI can test them without torch, a GPU, a 460 MB dataset or network access — the whole job runs in seconds. Four of those tests guard the CI badge itself against a workflow rename, which would otherwise blank it silently while tests kept passing.

11 Limitations

What this system does not establish.

- **Corpus scale.** 103,068 chunks from a streamed 10,000-row subset. Real deployments run orders of magnitude larger, and this project's own central finding — that tuned values do not survive scale changes — applies to its own conclusions.
- **The chunking ablation ran at a smaller scale** than the headline numbers (500 rows vs 2,000–10,000). Given three other values flipped with scale, the ordering of the top three strategies may too. What is unlikely to flip is semantic losing: it trails at every scale tested, at 10× the build cost.
- **Single language, single domain.** Hindi over MS MARCO web passages. Nothing here establishes behaviour on other Indic scripts or specialised domains.
- **Answer quality is measured by token F1** against a gold answer — a bag-of-words overlap metric that is blind to paraphrase. It rewards lexical similarity, not correctness.
- **The abstention negative set is n=150** real out-of-index queries. Larger and more adversarial sets would tighten the estimate.
- **No human evaluation.** Every quality number is automatic. Nobody read the answers and judged them.
- **Guardrail coverage cost is substantial.** The cross-encoder gate refuses 23% of answerable questions. For a demo that is a defensible trade; for production it would need work.

Future work

- Re-run the chunking ablation at 100k+ chunks to settle the ordering.
- Fine-tune the abstention gate rather than thresholding a similarity score — a small trained classifier over retrieval features would likely beat any single threshold.
- Streaming STT to overlap transcription with retrieval, cutting perceived latency.
- Quantise the embedder (int8) to widen deployment options.
- Human evaluation on a sample, to check what token F1 is missing.

12 Conclusion

What was built, and what it demonstrates.

The system works: speech in Hindi produces a grounded answer, retrieval holds a P50 of 11.54 ms and a P100 of 53.57 ms over 103,068 chunks, quality reaches MRR@10 0.551 on 1,219 gold-labelled queries, and the abstention gate cuts ungrounded answers from 100% to 14.7%. It is deployed and publicly reachable.

But the assembly is not the interesting part. Wiring an embedder to a vector index and a generation step is now a well-trodden path with good libraries. What distinguishes this build is that **the defaults were treated as hypotheses**, and most of them turned out to be wrong:

- The sophisticated chunker lost to the simple one, at 10× the cost.
- Hybrid retrieval — near-orthodoxy in RAG — lost to dense-only, and survived its own strongest counter-argument (RRF) before that was accepted.

- A guardrail documented as a safety feature was mathematically incapable of firing.
- Its replacement scored 0.93 on easy negatives and blocked 26% of realistic ones.
- Three separate tuned values reversed when the corpus grew 19×.
- A confident linear extrapolation was wrong by 3.7×.
- Three deploys reported success while shipping a Space that could not start.

The unifying lesson is narrow and portable: **a system that is not instrumented is a system whose failures announce themselves as successes**. Per-stage timing turned a latency regression into a two-minute diagnosis. A guardrails-on/off harness turned "we have safety features" into a number. Upload verification turned a green exit code into a caught error. In every case the instrumentation, not the feature, is what produced the knowledge.

The corollary is that **measurement is only as good as its adversary**. The guardrail calibrated on gibberish was measured, quantified, and reported at 0.93 — and worth almost nothing. A metric validated against easy cases is a number, not a guarantee.

13 Résumé and interview

How to present this, and how to defend it under questioning.

13.1 Résumé bullets

Lead with outcomes and keep every number paired with its scale — an MRR without a corpus size invites the exact follow-up you cannot answer.

COPY - PASTE

- Built and deployed a voice-enabled Hindi RAG pipeline (Sarvam STT → BGE-M3 dense retrieval over 103K FAISS-HNSW chunks → grounded generation) with query-time retrieval at 11.5 ms P50 / 53.6 ms P100, holding a sub-200 ms budget on a corpus 19× the initial build.
- Ran controlled ablations on gold-labeled MS MARCO-XI (chunking; dense/BM25/hybrid; RRF vs min-max fusion) and chose dense-only over hybrid on a negative result validated against its strongest counter-explanation.
- Profiled a retrieval regression to rank_bm25 consuming 99% of latency (superlinear degradation); swapped to a sparse-matrix BM25 for a 652× speedup at 0.998 rank correlation.
- Cut ungrounded answers from 100% to 15% with a cross-encoder abstention gate calibrated on realistic negatives; documented that abstention thresholds must be re-tuned per corpus size.
- Diagnosed five silent Hugging Face deploy failures (three exited 0 while failing), including a .gitignore rule that dropped the 430 MB index from the upload; shipped a one-command ZeroGPU deploy script.

13.2 What to emphasise, and why

Many candidates can assemble a RAG pipeline; the tutorials are good and the libraries do the hard parts. Far fewer can say *which of their defaults are wrong and how they found out*. That is the differentiator, and it is what these bullets are built around: three of the five describe diagnosis rather than construction.

FRAMING THAT WORKS

"I built a voice RAG system, and then I spent most of the time proving my own defaults wrong" is a far stronger opening than a feature list. It signals that you measure, and it sets up every follow-up question to land on ground you have actually covered.

13.3 Anticipated questions

"SUB-200 MS ON A VOICE PIPELINE? THAT SEEMS IMPOSSIBLE."

It is, end-to-end, and the report says so in its first section. The STT hop alone is 1.1–2.6 s — roughly 100× the local retrieval segment. The budget is scoped to query-time retrieval, which is the part actually under

engineering control, and both numbers are always reported together. *Volunteering this before being challenged is stronger than defending it after.*

"WHY DENSE-ONLY? HYBRID RETRIEVAL IS STANDARD."

It is, and it lost here. BM25 helped at 5,442 chunks and stopped helping at 21,657 — it weakens relatively as the corpus grows (MRR@10 0.259 vs dense 0.494). Before accepting that, I tested the likelier explanation: that min-max fusion was at fault, since it pins the best candidate at 1.0 however weak it is. RRF helped mid-range but still peaked at pure dense. Both were true — the fusion was mildly suboptimal *and* BM25 does not help this corpus. It stays wired in because the answer is corpus-dependent.

"HOW DO YOU KNOW THE GUARDRAIL ACTUALLY WORKS?"

Because I measured it with the gate on and off, on the same queries. The instructive part is that the first version scored 0.93 balanced accuracy against gibberish and blocked only 26% of realistic unanswerable queries — gibberish sits far from everything in embedding space and validates almost any threshold. Recalibrating on real out-of-index questions, then switching to a cross-encoder signal, took it to 85%.

"YOUR MRR OF 0.551 SEEMS LOW."

It is lower than the 0.606 I measured earlier on a smaller corpus, and that is the honest number. Quadrupling the haystack makes retrieval harder; retrieval metrics are a property of corpus size as much as retriever quality. I report the larger-corpus figure precisely because it is the credible one.

"WHAT WOULD YOU DO DIFFERENTLY?"

Scale earlier. Three tuned values reversed at 19× the corpus, so a chunk of Phase 2 tuning was done at a scale that did not represent the target. I would also have tested the LLM generation path directly instead of through its own fallback — graceful degradation hid a completely dead feature for weeks.

"WHAT WAS THE HARDEST BUG?"

The deploy that succeeded while dropping a 430 MB file. Exit code 0, "Committing 30/30" of 31 files. The cause was our own `.gitignore` — `huggingface_hub` feeds it to the preupload API, and it carried `*.faiss` to keep build artifacts out of git. Correct for the repository, wrong for the Space, where the index is the payload. I fixed it wrongly twice — a missing LFS pattern, then commit ordering — both plausible, both untested, before reading the library source and finding the real resolution order.

"WHY NOT USE A VECTOR DATABASE LIKE QDRANT?"

Considered and rejected on measurement. FAISS HNSW searches 103k vectors in 0.57 ms and metadata filtering already exists in the retriever. Qdrant would have added a Docker dependency and a day of work to gain a keyword. At a scale that outgrows one machine the answer changes.

13.4 Questions to ask them

Turning the ablation habit outward reads as genuine and is hard to fake:

- "How do you decide when a retrieval change is actually an improvement — what's the eval set?"
- "Do your abstention or safety thresholds get re-tuned when the corpus grows?"
- "What's your ratio of measured defaults to inherited ones?"

THE ONE THING TO REMEMBER

Every claim in this report is reproducible from the repository, and the failures are documented alongside the wins. That is the posture worth carrying into the room: not that the system is impressive, but that **you know exactly what it does, what it does not, and how you established the difference.**

Voice RAG Field Report · Every figure reproducible from `benchmark.py`, `eval.py` and `scripts/faithfulness.py`. Corpus: 103,068 chunks, MS MARCO-XI Hindi. Hardware: RTX 4060, 8 GB.